

EMBEDDED SYSTEM AND DESIGN

EXPERIMENT NO. 7

REAL-TIME VAULT SYSTEM

Name: *Raphael S. Nano*

Course/Section: *CPE161P / AI*

Date of Performance: *January 11-12, 2023*

Date of Submission: *January 12, 2023*

Ma'am Cyrel Manlises
Instructor

DISCUSSION

Vault systems are examples of structures that are designed to secure and protect valuable assets and confidential information. These can be in a physical form or digital form or an integration of both, wherein in the integrated realm, vaults are fortified enclosures with reinforced doors that are protected with an access system, which can be in the form of keypads, biometrics, RFID, and many more. These systems play a crucial role in maintaining the confidentiality, integrity, and availability of critical information. Considering that, a real-time vault system was created, wherein the operations involve keypad as the input password, LCD for the output of the lock's status, and the servo motor to open and close the vault itself.

For experiment 7, the objectives are as follows:

- 1. To demonstrate how a real-time vault system works.*
- 2. To be able apply what you have learned in logic circuits and switching theory, microprocessors, and CPE160P.*
- 3. To be able to apply real-time embedded system using Arduino kit.*

The materials involved in constructing the vault system are the Arduino Uno, the 4x4 matrix keypad, Liquid Crystal display, and servo motor.

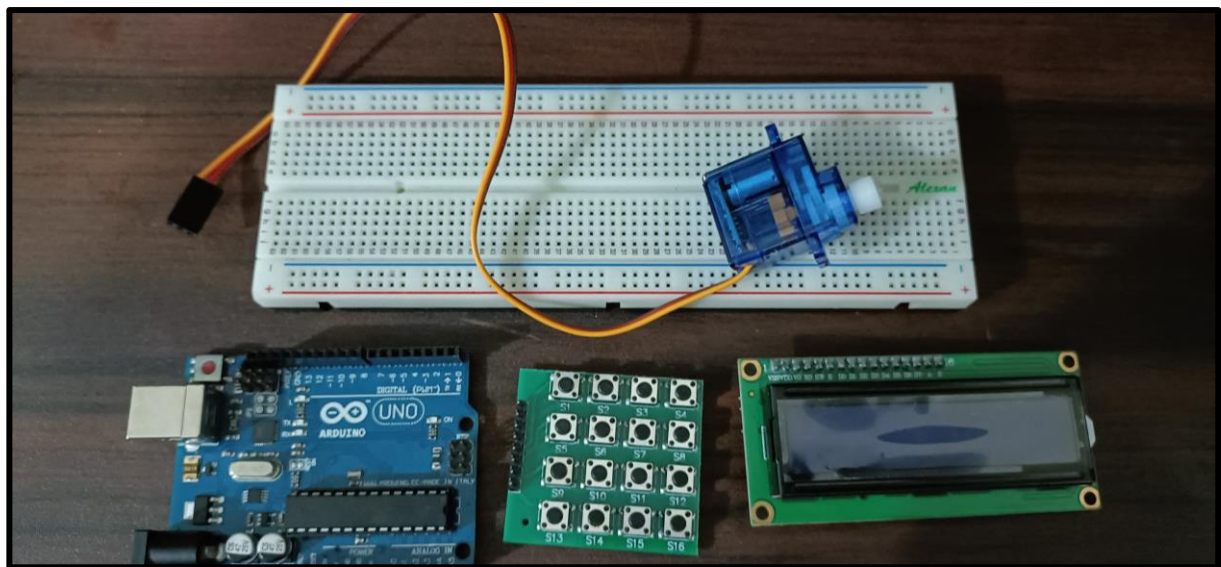


Figure 1. Major Components for the Experiment

For the external libraries, Keypad.h was utilized for the switches, LiquidCrystal_I2C.h was for the LCD, and Servo.h was included for the servo motor.

```
#include <Keypad.h>
#include <LiquidCrystal_I2C.h>
#include <Servo.h>
```

Figure 2. External Libraries Included

The primary aim of experiment 7 is to create a real-time vault system, wherein a set 6-digit password should be the input of the user in order to open the vault for 20 seconds. A button may also be pressed so that the time would be increased by 10 seconds. An "Unlocked" and "Incorrect" statuses can be seen, depending on the input of the user.

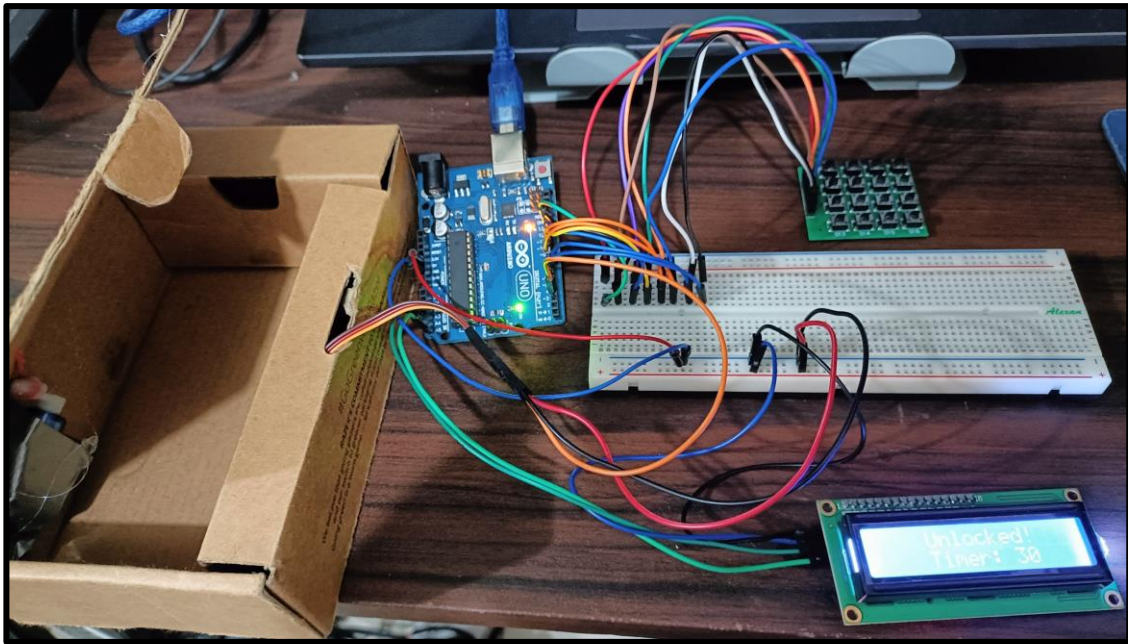


Figure 3. Final Output of the Experiment

The main component of this experiment is the keypad. By using the getKey() predefined function from Keypad.h library, the user input was made into an array so that the series of numbers will be stored individually and concatenated at the end. If the data input is already equal with the length of the password, you can no longer input numbers.

```
customKey = kp.getKey();
if (customKey) {
  if (dataNum != PW_len) {
    userIn[dataNum] = customKey;
    lcd.setCursor(dataNum, 1);
    lcd.print(userIn[dataNum]);
    dataNum++;
  }
}
```

Figure 4. Keypad Main Function

Moreover, it was made that the "#" symbol will be the ENTER key. That will be the last input of the user to test if the input is correct. Using strcmp, the set password will be compared with the user input. If the result is 0, the vault will be unlocked, so, the lcd prints "Unlocked" and the servo motor moves to angle 0 to open the vault. On the other hand, if the result is not 0, the vault will remain locked.

```
if (userIn[dataNum - 1] == '#') {  
    lcd.clear();  
    dataNum--;  
    userIn[dataNum] = 0;  
    if (!strcmp(userIn, mainPW)) {  
        lcd.setCursor(4, 0);  
        lcd.print("Unlocked!");  
        servo.write(0);  
        digitalWrite(signalPin, HIGH);
```

```
    else {  
        lcd.setCursor(3, 0);  
        lcd.print("Incorrect!");  
        delay(2000);
```

Figure 5. Lock Functions

For the timer, a while loop was used, wherein as long as i is greater than 0, the i counts down and the LCD continues to print the value of the timer and the word unlocked.

```

while (i > 0) {
  lcd.setCursor(4, 0);
  lcd.print("Unlocked!");
  if (i < 10) {
    lcd.setCursor(4, 1);
    lcd.print("Timer: ");
    lcd.setCursor(11, 1);
    lcd.print("0");
    lcd.print(i);
    msdelay(1000);
    lcd.clear();
  } else {
    lcd.setCursor(4, 1);
    lcd.print("Timer: ");
    lcd.print(i);
    msdelay(1000);
    lcd.clear();
  }
  i--;
}

```

Figure 6. Loop function for the Timer

If the "" was pressed while counting down, the value of i will be increased by 10 to extend the time that the vault is open. This function can be seen in the msdelay() function, wherein it checks if the "*" is pressed once.*

```

void msdelay(int delNum) {
  int startms = millis();
  int currentms = millis();
  int addFlag = 0;
  while (currentms - startms < delNum) {
    currentms = millis();
    add = kp.getKey();
    if ((add == '*') && (addFlag == 0)) {
      i = i + 11;
      addFlag = 1;
    }
  }
}

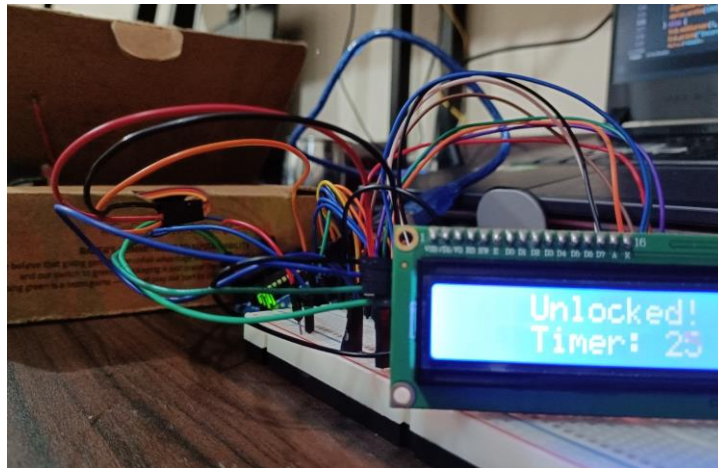
```

Figure 7. msdelay() with adding of timer

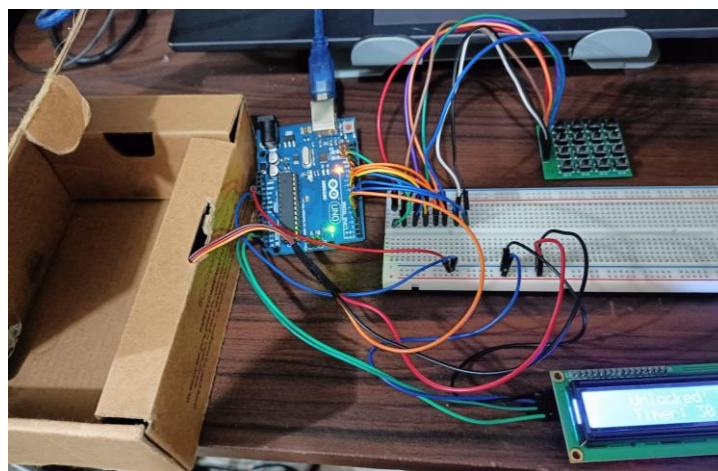
Lastly, to clear the data, all of the characters inside the userIn array will be removed to reset the program, the value of i will be back to 20 and dataNum is also set to 0.

```
void clearData() {  
    while (dataNum != 0) {  
        userIn[dataNum] = 0;  
        dataNum--;  
    }  
    userIn[dataNum]=0;  
    dataNum = 0;  
    i = 20;  
    return;  
}
```

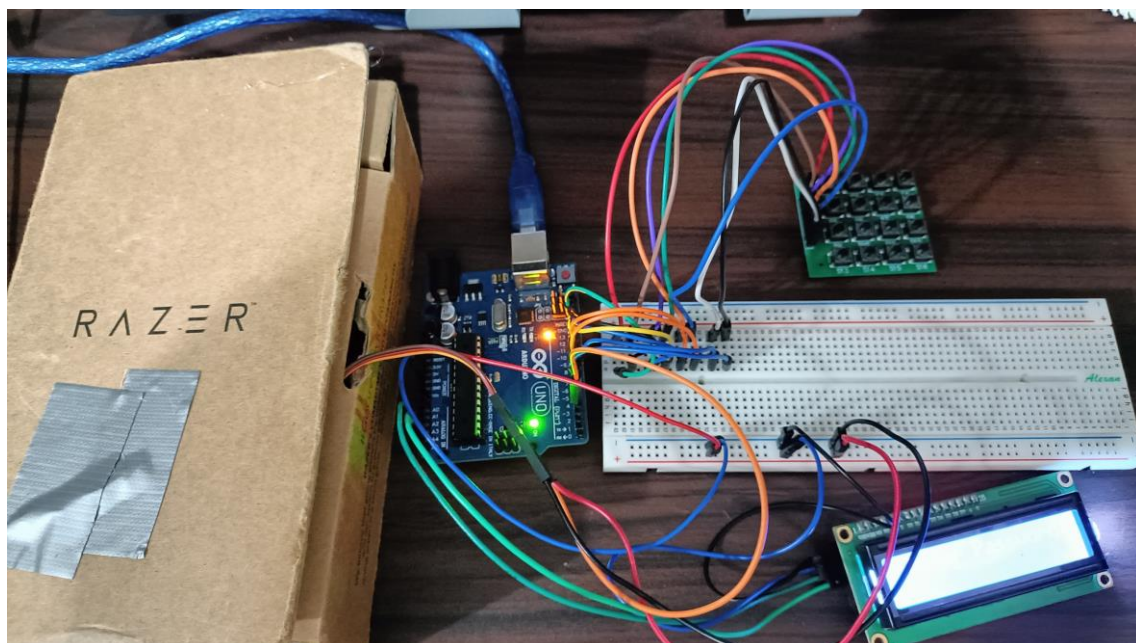
Figure 8. Function for clearing data



(a)



(b)



(c)

Figure 9. Final Out put of Experiment 7

CONCLUSION

Overall, the real-time vault system operates in three main functions: (1) the input of the user in the keypad, (2) the comparing of the input and the set password, which can be seen on the LCD, along with the timer and status, and (3) the opening and closing of the vault by the servo motor. The second objective was achieved by synchronizing the keypad with the LCD and servo motor, demonstrating the usage of microprocessors, logic circuits, switching theory, and CPE160P lessons. The real-time vault system was successfully built using the Arduino kit, and basic handshaking was shown by having synchronization in the input and output in real time.